

2 class Node:

```
def __init__(self, data):  
    self.data = data  
    self.prev = None  
    self.next = None
```

class DoublyLinkedList:

```
def __init__(self):  
    self.head = None
```

```
def insert_front(self, data):  
    node = Node(data)  
    if self.head:  
        self.head.prev = node  
        node.next = self.head  
    self.head = node
```

```
def insert_end(self, data):  
    node = Node(data)  
  
    if self.head is None:  
        self.head = node  
        return
```

```
temp = self.head  
while temp.next:  
    temp = temp.next
```

```
temp.next = node
```

```
node.prev = temp
```

```
def delete(self, key):
```

```
    temp = self.head
```

```
    while temp:
```

```
        if temp.data == key:
```

```
            if temp.prev:
```

```
                temp.prev.next = temp.next
```

```
            else:
```

```
                self.head = temp.next
```

```
            if temp.next:
```

```
                temp.next.prev = temp.prev
```

```
            return
```

```
        temp = temp.next
```

```
    print("Element not found")
```

```
def display(self):
```

```
    temp = self.head
```

```
    while temp:
```

```
        print(temp.data, end=" <-> ")
```

```
        temp = temp.next
```

```
    print("None")
```

```
def count(self):  
    c = 0  
    temp = self.head  
    while temp:  
        c += 1  
        temp = temp.next  
    print("Count =", c)
```

```
def search(self, key):  
    temp = self.head  
    while temp:  
        if temp.data == key:  
            print("Found")  
            return  
        temp = temp.next  
    print("Not Found")
```

```
dll = DoublyLinkedList()
```

```
while True:  
    print("\n1.Insert Front")  
    print("2.Insert End")  
    print("3.Delete")  
    print("4.Display")  
    print("5.Count")  
    print("6.Search")  
    print("7.Exit")
```

```
ch = int(input())
```

```
if ch == 1:
```

```
    dll.insert_front(int(input("Enter element: ")))
```

```
elif ch == 2:
```

```
    dll.insert_end(int(input("Enter element: ")))
```

```
elif ch == 3:
```

```
    dll.delete(int(input("Delete element: ")))
```

```
elif ch == 4:
```

```
    dll.display()
```

```
elif ch == 5:
```

```
    dll.count()
```

```
elif ch == 6:
```

```
    dll.search(int(input("Search element: ")))
```

```
elif ch == 7:
```

```
    break
```

```
3. stack = []
```

```
while True:
```

```
    print("\n1.Push")
```

```
    print("2.Pop")
```

```
    print("3.Display")
```

```
    print("4.Exit")
```

```
ch = int(input("Enter choice: "))
```

```
if ch == 1:  
    stack.append(int(input("Enter element: ")))
```

```
elif ch == 2:  
    if stack:  
        print("Popped:", stack.pop())  
    else:  
        print("Stack Underflow")
```

```
elif ch == 3:  
    if stack:  
        print("Stack:", stack)  
    else:  
        print("Stack Empty")
```

```
elif ch == 4:  
    break
```

```
4. queue = []
```

```
while True:  
    print("\n1. Insert")  
    print("2. Delete")  
    print("3. Display")  
    print("4. Exit")
```

```
ch = int(input("Enter choice: "))
```

```
if ch == 1:  
    queue.append(int(input("Enter element: ")))
```

```
elif ch == 2:  
    if queue:  
        print("Deleted:", queue.pop(0))  
    else:  
        print("Queue Underflow")
```

```
elif ch == 3:  
    if queue:  
        print("Queue:", queue)  
    else:  
        print("Queue Empty")
```

```
elif ch == 4:  
    break
```

5. SIZE = 5

```
queue = [None] * SIZE
```

```
front = -1
```

```
rear = -1
```

```
while True:  
    print("\n1. Insert")  
    print("2. Delete")  
    print("3. Display")  
    print("4. Exit")
```

```
ch = int(input("Enter choice: "))
```

```
if ch == 1:
```

```
    val = int(input("Enter element: "))
```

```
    if (rear + 1) % SIZE == front:
```

```
        print("Queue Overflow")
```

```
    elif front == -1:
```

```
        front = rear = 0
```

```
        queue[rear] = val
```

```
    else:
```

```
        rear = (rear + 1) % SIZE
```

```
        queue[rear] = val
```

```
elif ch == 2:
```

```
    if front == -1:
```

```
        print("Queue Underflow")
```

```
    else:
```

```
        print("Deleted:", queue[front])
```

```
        if front == rear:
```

```
            front = rear = -1
```

```
        else:
```

```
            front = (front + 1) % SIZE
```

```
elif ch == 3:
    if front == -1:
        print("Queue Empty")
    else:
        i = front
        print("Queue:", end=" ")
        while True:
            print(queue[i], end=" ")
            if i == rear:
                break
            i = (i + 1) % SIZE
        print()
```

```
elif ch == 4:
    break
```

6. from collections import deque

```
dq = deque()
```

```
while True:
    print("\n1. Insert Front")
    print("2. Delete Front")
    print("3. Insert Rear")
    print("4. Delete Rear")
    print("5. Display")
    print("6. Exit")
```



```
ch = int(input("Enter choice: "))
```

```
if ch == 1:
```

```
    dq.appendleft(int(input("Enter element: ")))
```

```
elif ch == 2:
```

```
    if dq:
```

```
        print("Deleted:", dq.popleft())
```

```
    else:
```

```
        print("Deque Empty")
```

```
elif ch == 3:
```

```
    dq.append(int(input("Enter element: ")))
```

```
elif ch == 4:
```

```
    if dq:
```

```
        print("Deleted:", dq.pop())
```

```
    else:
```

```
        print("Deque Empty")
```

```
elif ch == 5:
```

```
    print(list(dq))
```

```
elif ch == 6:
```

```
    break
```